

# SANKUR KUNDU

## Software Engineer | Backend, Systems & Full-Stack

Kolkata, India (open to relocation) | +91 97761 94201 | [sankur.kundu.tw@gmail.com](mailto:sankur.kundu.tw@gmail.com)  
[linkedin.com/in/sankur-kundu](https://linkedin.com/in/sankur-kundu) | [github.com/Shakhtar-Sankur](https://github.com/Shakhtar-Sankur) | [shakhtar-sankur.github.io](https://shakhtar-sankur.github.io)

## SUMMARY

Software engineer (B.Tech CSE, 2026) who builds systems from scratch and proves them correct: a fault-tolerant LLM cluster in C++/CUDA, RL post-training (GSM8K 48% → 52.5%), a TLA+-verified SQL database. Sole engineer at Gigzen.

## PROJECTS

### relay — C++, CUDA and Swift: a disaggregated, fault-tolerant LLM inference cluster

- Built C++/CUDA prefill and decode workers on my own flash-attention and flash-decoding kernels, a KV-cache transfer engine that streams each layer while the next computes, and a Swift control plane with prefix-aware routing.
- On a T4, cut 2,048-token prefill **8.7×** (3.7 s → 0.43 s) and batch-32 decode **5.4×**; streaming hides **99–100%** of the KV transfer. Output matches Hugging Face and stays bit-identical when batched, cached or split across workers.
- Fault-tolerant: watchdogs catch killed, frozen or stalled workers and their requests resume elsewhere with identical output (384/384 through 60 kills; no request lost in 300 fault-injection seeds).

### ratchet — Python, PyTorch and C++/CUDA: reinforcement-learning post-training (GRPO) from scratch

- Built GRPO post-training end to end: rollouts with per-token log-probabilities on relay (through a C API), a PyTorch trainer data-parallel on tandem, in-place GPU weight sync, partial rollouts and one-step-ahead training.
- On 2× T4, lifted Qwen2.5-0.5B on GSM8K from **48.1% to 52.5%** in 100 steps, with weight sync **369×** faster than a reload; found and fixed fp16 rounding that silently erased training updates (log-prob gap **5.2 → 0.04**).

### quorumdb — Rust, zero dependencies: a distributed SQL database, PostgreSQL wire-compatible

- Built a distributed SQL database that psql connects to: LSM storage, Raft and self-rebalancing multi-Raft, serializable ACID transactions (Percolator) and a SQL planner; **1,500 tpmC** on a TPC-C-derived benchmark.
- Verified by deterministic simulation (**10,000** Raft clusters, 610K crashes, histories machine-checked) and a TLA+ model (337K states): found **6 real bugs** and catches all 15 planted ones.

### Also built

- switchyard**: mixture-of-experts serving across 2 GPUs on my own CUDA kernels, **2.2×** Hugging Face.
- tandem**: distributed training (DDP, ZeRO-1/2/3, 1F1B) **bit-identical** to PyTorch on 2× T4.
- tessel**: a Triton-style tile language for GPUs and TPUs; GEMM at **cuBLAS parity**, an LLM engine at **1.05×** vLLM.
- faultline**: Jepsen-style fault injection across 4 etcd release lines (**1.26M** operations under 288 faults).
- kiln**: an ML compiler **faster than torch.compile** on CPU; **ferrolm**: an LLM server (**3.4×** throughput from batching).

## OPEN SOURCE

- PyTorch **#199875** (under review): torch.compile now fills torch.empty in deterministic mode as eager does, without filling Inductor's internal buffers, the cause of an earlier fix's revert.
- PyTorch **#199441**, **#199646** (under review): removed **13** stale torch.compile test skips, verified on CPU and 2× T4; **8** skipped determinism tests now run under torch.compile.

## EXPERIENCE

### Gigzen — Co-founder & Founding Engineer

Apr 2026 – Present

Early-stage startup (incorporation in progress) · Sole engineer across every product

- Sole engineer on **Waggle**, a delivery platform of three Android apps (riders, shops, customers) from one TypeScript/React/Capacitor codebase on Supabase PostgreSQL: **53 tables**, **90 RLS policies**, **154 SQL functions**.
- Put the business rules in the database, where no client can bypass them: first-accept-wins dispatch, 4-digit handover codes, Razorpay/UPI payments reconciled by UTR, GST invoices and pg\_cron billing.
- Security-tested it with **150 automated attacks** on its database access rules (**all blocked**) and a 42-step Playwright run.
- Built **Populace**, an open-source multi-user testing tool (npm CLI, GitHub Action, Electron app); its first run found **5 defects** in **3.5 minutes** after manual testing had passed, one breaking every signup.

## SKILLS

**CS Fundamentals:** Data Structures & Algorithms, System Design, Distributed Systems, DBMS internals, OS, Networks

**Languages:** C++, Rust, Python, Swift (C++ interop), TypeScript, JavaScript, SQL (PL/pgSQL), Java

**Backend & Data:** PostgreSQL, Supabase, Row-Level Security, pg\_cron, REST APIs, Node.js, FastAPI, Flask, Redis, Razorpay/UPI

**Frontend & Mobile:** React, Capacitor (Android), Electron, Leaflet/MapLibre maps, HTML, CSS

**Systems & performance:** CUDA (tensor cores), LLM serving, MoE expert parallelism, RL post-training, DDP/ZeRO, SIMD

**Testing, DevOps & Cloud:** Simulation testing, fault injection, TLA+, Playwright, GitHub Actions, Docker, Kubernetes, GCP

## EDUCATION

B.Tech, Computer Science and Engineering (AI & ML) — IEM Kolkata

Sep 2022 – Apr 2026